Project: ''Dynamic'' Traffic Lights

Student

Beni Cherniavsky

Project Supervisor

Ari Todtfeld

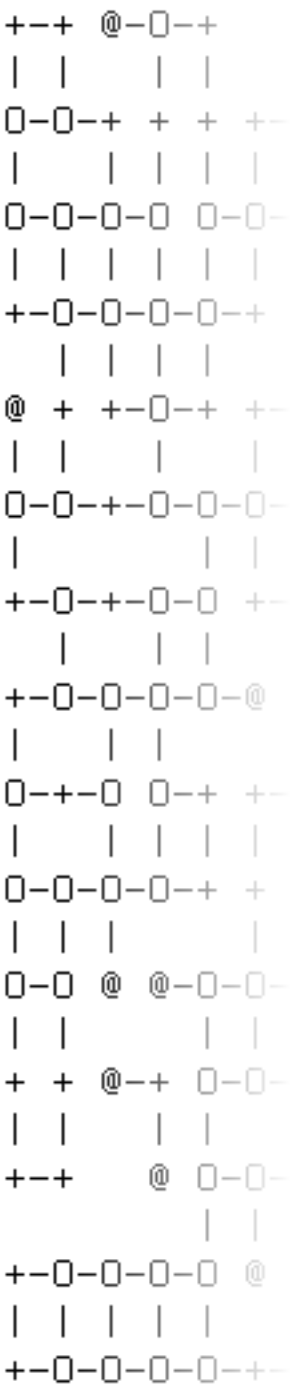
Lab

Control, Robotics and Learning Systems Lab, Electrical Engineering, Technion, Israel

Overview

This project analyzed systems of traffic ligths, roads and cars, in search of the best startegy for controlling the traffic lights.

- Problem formulation.
- Selection of a (quite simplified) model.
- Simulation software was developed for this model and run with many control strategies (due to time constraints only simple strategies were tried).
- Analysis of the the results.



Part 1 - Problem formulation

In one sentence: given junctions, roads and cars find the best strategy for controlling the junctions.

- Definition of "best" will come later.
- Since the project was open-ended, it was needed to decide on the level of complexity with which to work. The problem eventually analyzed was, unsurprisingly, very much of a toy problem.

The (too) big problem

Let's first look at the most general (more or less) formulation of the problem to see what was omitted in the toy problem. Our world consits of three objects:

- Car that move around. They can probably enter / leave the city at certain places. At junctions they decide what turns they should take. They can prefer some roads to others, for complex reasons.
- Unidirectional roads that delay cars driving through them. The delay can depend on factors like amount of cars, random effects, etc.
- Junctions -- our question is *How to control them best?* They are the most complex part.

Real junctions

- A given junction has *M* input roads and *N* output roads. There are *M*N* routes for a car to pass through a junction (or less).
- A car arriving from input direction *m* and decides it wants to go to direction *n* are appended to queue $Q_{m,n}$ of the junction (not necessarily distinct).
- At any moment the junction is serving zero or more queues. "Serving" means that from time to time (not necessarily at a fixed rate), one car (or more), is popped from the queue (unless it's empty) and forwarded to the target road(s) the car had chosen.

Our task is to choose which queue(s) to serve at every moment.

Serving constraints

- Avoid collisions. Only some combinations of routes through the junction can be safely served at the same time.
- switching rate. Between serving different sets of queues we most waste some more time to avoid collision. Let's call all this dead time "yellow light" for simplicity.
- Pedestrians deserve green time too. Ignored in this project :-).

What do we want?

By what criteria should we measure how ``good'' a given control policy is?

- Most importantly, we want minimum total **delay** for a car along his way.
- We also want **stability** (predictable delays are better than strongly fluctuating ones) and **fairness** (different cars should get similar treatment if possible).
- Cars travel in different ways => we measure delays per queue.
- Stability/fairness can be represented by the variance of the delay among cars. There are subtle points in these definitions - over what sets we take the means/variances?

Sensors / inputs

What input should be available to the control system? One can divide the research in this field into three somewhat separate parts:

- Given perfect information, develop the optimal control strategy.
- Check how sensitive the strategy is to imperfect information.
- Devise cheapest sensors giving maximum useful information.

We do the first part, so let's help ourselves to whatever information we need :-).

Down to earth - the toy problem

That's too complex. Let's make lots of assumptions:

- Cars decide where to turn statistically with fixed known probabilities, e.g. 60% straight, 20% right, 15% left, 5% reverse. Bad model of reality but convenient for us (equal arrival rate at all junctions is easy to arrange).
- This means that all roads are equal. Also, let all queues be equal, serving one car per time unit.
- So let's also adopt discrete time - simplifies simulation and especially control definition.
- Every road has a constant delay and a "vertical" queue at the end.
- No yellow light time. We do check the impact of imposing a lower limit on the time each queue is served.

Down to earth - the toy problem (cont.)

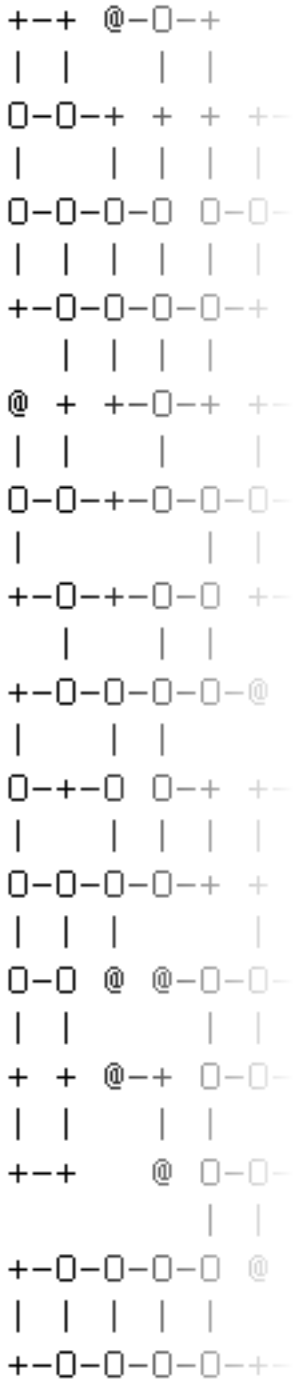
And now the most dramatic limitation:

- Serve only one queue at a time. Vastly simplifies control decisions.
- This is far from reality. Most importantly, this limits the load on each queue to 0.25 (assuming 4 roads per junction)!

Part 2 - Theoretical analysis

I first attempted to an analytical approach. I considered a single junction:

- Without any sensors (can try to serve empty queues).
- Arrival rates are given.
- Choose a random queue to serve on each cycle.
- I easily found the optimal serving probabilities.



First lesson - don't serve empty queues

I tried to consider more complicated problems.

- For example to check which queues are empty and avoid serving them.
- This already is **much** harder to analyze (the queues are no longer independent).
- But a quick calculation showed that **more than 50%** of the time can be wasted on empty queues!
- I can't analyze anything but uselessly simple problems. Time for empirical work - I need a simulator to make experiments.

Part 3 - Software

The heaviest and longest part of the project was actually writing the simulation system. But this less interesting to present so I'll make it short :-).

More details about the architecture can be found in the full report, the documentation extracted from the code and the source code itself.

Python

I chose **Python** as my implementation language.

> Python is an interpreted object-oriented programming language suitable (among other uses) for distributed application development, scripting, numeric computing and system testing. Python is often compared to Tcl, Perl, Java, JavaScript, Visual Basic or Scheme. To find out more about what Python can do for you, point your browser to http://www.python.org/.

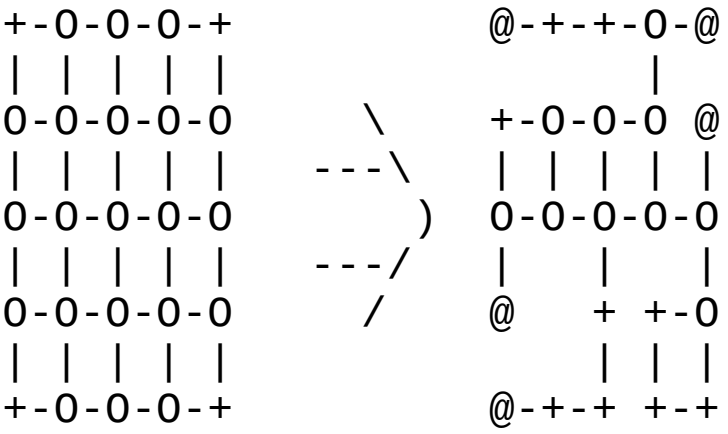
Was it worth it? Definitely yes!

- Quick prototyping; I didn't feel that debugging takes most of the time...
- The definition of the simulation batch runs and their processing is what scripting languages are ideal for.
- This presentation was created with two Python tools.

City generation

- A Manhattan-style geometry.
- For more interest, strip 30% of the roads.
- With 3 and 4 roads, it's a junctions.
- With 2 roads, it's a long road continuation.
- With 1 roads, it's car source/drain.

Like this:



Control specification

To implement a wide range of control strategies, a pipeline of plug-in filters was used, where each filter takes a list of queues considered for serving ad returns a new list (usually subset).

- This made specifying all desired control strategies surprisingly easy.

This was further abstracted as orthogonally choosing several parameters, specified as strings:

- free / hold*n* - switching rate
- fix / flex - adaptiveness of switching rate
- One of pricipal control strategies - see next slide
- rand / lru - tie breaking

Control specification (cont.)

The principal control strategies are:

blind
: Completely dumb. Decided by rand / lru.

nonempty
: Similar but don't serve empty queues.

longest
: Serve longest queue.

inorder
: Serve in order of arrival

hist
: Serve queue predicted to empty last, according to measurements or load history.

Part 4 - Simulations

There were two serious runs:

- The first was intended to probe various pararmeters to get some feeling and determine interesting parameters for the real run.
- With these parameters a much heavier simulation was run to provide high-quality data for the analysis.

Probing the ground

The first thing I wanted to check was how long I need to simulate and on how big a city, to get valid averaged results.

- I tried three city sizes and built graphs of total number of cars, cumulative mean delay and variance of delay, as a function of the simulation time; car sources were first on and after 4000 cycles were turned off.
- The conclusions was that 15x15 and 5000 cycles are good.
- The collected statistics are instructive on their own.

See the LaTeX report for the nice numerical tables.

Conclusions

All non-blind strategies have almost equal delay (with unconstrained switching).

- Least-Recently-Used is **much** better than random serving. The magnitude of the difference surprised me. This trend repeated in all following runs.
- In-order serving gives lower variance than all other. This was expected.
- The load history measurement doesn't differ noticeably from just serving the longest queue.

When serving each queue for 4 cycles a time, the results are similar.

- For very low loads, serving in-order also gives lower delay!

Heavy car crunching

A more exhaustive simulation was run, with the following parameters:

- 15x15 cities.
- 3 different city plans were simulated in parallel.
- 5000 cycles.
- Higher loads: 0.15, 0.20, 0.23.
- More control policies. Switching rates: free / hold2 / hold4 / hold8, fixed / adaptive.
- Separate statistics for junctions with 3 and 4 roads, since they experience different loads.

This amounted to 648 runs, 207,140,557 car-junctions and above 48 hours of total run time :-).

Results

To display the full data I need an 8-dimensional table... So I wrote a python script to take various cuts/projections of it, reorder the axes and create nicely-looking LaTeX tables.

First, I compared between junctions with 3 and 4 roads.

- As suspected, there is a big difference. So for precise results, I didn't averaged between them any longer.

Then I compared the 3 city plans, for same parameters.

- There are very noticeable and quite systematic differences.
- For lower loads, the delays are always A > B > C. For higher loads, B > A ~= C. In all cases B has the highest deviation.

Results (cont.)

- What in the geometry controls this? I found only one thing: B had much more car sources/drains; they emit cars with lower auto-correlation than junctions, and this interferes with the control policy.
- So can we trust the other results, if so much depends on the city geometry? Fortunately yes, because relations between the values are approximately the same for all plans.

Now let's compare the effect of the switching rate.

- If the switching rate is limited, adaptive limiting gives lower delay than fixed one. This makes sense.
- On the other hand, it can harm the fairness.

Results (cont.)

Last, I compared the different control policies.

- Basically, the results agree with the first run.
- With the increase in the load, the differences aren't smaller. It could be expected that when all queue are full, a blind policy won't have a higher delay. Probably the load is not high enough.
- With slow switching rates, the policies do become similar. None of them can predict the future very well.
- As previously, serving in-order gives the lowest deviation but the delay isn't as low as when serving the longest queue.

The bottom lines

- First, avoid serving empty queues as much as possible. Choosing the longest queue is a good strategy to minimize this. Breaking the tie by choosing the one whose front car arrived first is better yet.
- Second, flexibly controlling the green time is another way to reduce it. Serve a queue as long as it has cars, when it empties, switch to another.
- Third, always prefer least-recently-used approaches to picking at random. This helps both for delay and fairness.
- There is some trade-off between delay and fairness. Serving in order of arrival is always most fair; however it is not very practical since if the switching rate is limited, it gives suboptimal delay.

Future directions

In this project I didn't get really interesting results; it was more of a foundation from which somebody can continue. There are many ways.

- One could perform more research with the current simulator. The conditions tried are not representative of real-life traffic; higher loads, longer green times, longer roads and yellow lights would be interesting to explore. The role of the city's geometry should be explored more; the current Manhattan-like geometry is very arbitrary.
- Implement logging/visualization facilities that would allow one to track the history of a single car (or a little group) and get a feeling of the system's behavior. This could help to gain intuition for second-order effects.

Future directions (cont.)

- Smarter car routing; a simple approach is to choose a route like now when a car is created and remove all cycles from it.
- Account for main/local roads; make cars prefer big roads.
- Each of the assumptions described at the first chapter could be lifted. Serving multiple queues at once would be most valuable for getting closer to reality.
- The ultimate goal: devise control policies that involve communication between junctions.
- Use your imagination to continue this list...